



Nombre de la materia

Calidad del software II.

Nombre del programa

Ingeniería de software

Nombre del estudiante

MATÍAS VIRGÜEZ

Nombre de la actividad

Diseñemos y ejecutemos las pruebas de software.

Unidad #

Unidad 2 Actividad 4

Nombre del profesor

JEFERSON URREGO GUERRERO.

Fecha

26-09-2026

Dashboard de KPIs de rendimiento

Sistema bajo prueba: QuickPizza (quickpizza.grafana.com)

LINK:

<https://github.com/Matiasv18/prueba-rendimiento-quickpizza/tree/main/dashboard>

Dashboard y fuentes de datos

El dashboard se construyó en Grafana (instalación local) sobre dos fuentes CSV que no son datos de ejemplo sino los resultados verificables de la Actividad 3: resultados_pruebas.csv (percentiles P50/P95, tasa de error y throughput de las 3 pruebas) y observabilidad_soak.csv (CPU/RAM del cliente en 3 momentos de la prueba de resistencia). Esta decisión de diseño importa porque cada cifra que aparece en un panel puede rastrearse hasta un reporte HTML o una captura concreta, lo cual es justamente el estándar de trazabilidad que exige un dashboard operativo y no solo una ilustración. La estructura resultante son 5 paneles: dos de latencia P95 con umbrales de SLA ya configurados, uno de observabilidad de recursos, y dos adicionales (throughput y error) que, aunque no se explican en detalle en este comentario, sostienen visualmente la lectura de los dos KPIs elegidos. El export en JSON, las capturas PNG y el repositorio versionado completan la evidencia adjunta.

KPI 1 — Latencia P95 en POST /api/pizza

Elijo este KPI en primer lugar porque no todos los endpoints pesan igual: home entrega contenido estático y su latencia dice poco sobre el sistema real, mientras que /api/pizza ejecuta lógica de negocio (el motor de recomendación), así que es ahí donde una eventual degradación tendría consecuencias sobre la experiencia del usuario. El dashboard registra un P95 de 659.59 ms en carga, 672.54 ms en estrés y 686.84 ms en resistencia, en los tres casos muy por debajo del SLA de 2000 ms definido en la Actividad 3. Lo que hace interesante a este dato no es que cumpla el umbral —eso ya lo sabíamos desde el informe anterior— sino que prácticamente no se mueve al pasar de 15 a 100 VUs concurrentes. Si el servidor estuviera cerca de su límite de capacidad, esperaríamos ver una cola creciente reflejada en un P95 que se dispara con la concurrencia; como eso no ocurre, la lectura correcta es que el sistema opera con margen holgado en todo el rango probado, y que el riesgo de incumplir este SLA específico es, por ahora, bajo. Esto es consistente con el enfoque de Majors et al. (2022), quienes insisten en que un buen panel debe priorizar señales que reflejen la experiencia real del usuario por encima de métricas de infraestructura que no necesariamente se traducen en impacto percibido.

KPI 2 — Throughput (solicitudes por segundo)

El segundo KPI es más revelador precisamente porque, a primera vista, parece un problema: el SLO exigía ≥ 15 req/s y el dashboard muestra apenas 7.60 req/s en carga y 6.13 req/s en resistencia, cumpliéndose solo en estrés (18.43 req/s). La pregunta que vale la pena hacerse no es si se cumplió el número, sino por qué no se cumplió, y ahí es donde el análisis se vuelve útil. El script modela un think

time de 1 a 4 segundos entre acciones, lo que produce una iteración promedio de ~ 4.85 segundos; dado que cada iteración genera 2 solicitudes, el techo matemático de throughput por VU es de apenas ~ 0.41 req/s, independientemente de qué tan rápido responda el servidor. Multiplicado por 15-20 VUs, ese techo cae justo en el rango de 6-8 req/s que efectivamente se observó. En otras palabras, el patrón no es una cola de saturación del Backend sino un límite estructural impuesto por el propio diseño del modelo de carga, y la prueba de esto es que la latencia (KPI 1) no se degrada mientras el throughput sube junto con los VUs en estrés. Confundir este hallazgo con una falla de capacidad llevaría a invertir esfuerzo de optimización donde no hace falta.

Recomendaciones priorizadas

De los dos KPIs se desprende una misma conclusión: el sistema no muestra señales de riesgo de capacidad en el rango evaluado, pero el criterio con el que se definió el SLO de throughput sí necesita revisión. Siguiendo el criterio de la Google SRE Workbook de actuar solo sobre señales que reflejan riesgo real (y no perseguir cada desviación numérica por igual), priorizo así:

#	Recomendación	Impacto	Complejidad
1	Recalibrar el SLO de throughput por perfil de prueba, ajustándolo a la concurrencia realmente modelada en cada escenario.	Alto	Baja
2	Extender la prueba de estrés más allá de 100 VUs para ubicar el punto de quiebre real del servicio.	Medio	Media
3	Instrumentar tracing distribuido en el Backend (siguiendo Parker et al., 2020) para vincular la latencia observada con componentes específicos de la arquitectura.	Alto	Alta

Ordeno estas tres acciones por lo que realmente cuestan frente a lo que realmente resuelven, no por su apariencia técnica. La primera no toca una sola línea de infraestructura: es reconocer que el número "15 req/s" se definió sin mirar el think time modelado, y corregirlo es un ejercicio de criterio, no de ingeniería. Las otras dos sí exigen trabajo real, pero por razones distintas: extender el estrés más allá de 100 VUs responde a una pregunta que este dashboard deja abierta —¿dónde está el límite, si es que existe en un rango razonable?—, mientras que instrumentar tracing distribuido (Parker et al., 2020) ataca una limitación de fondo: todo lo que este dashboard mide proviene del cliente, así que por más paneles que se agreguen, nunca podrá confirmar por sí solo qué ocurre dentro del servidor cuando la latencia se acerca a su umbral. Priorizar la recomendación 1 no es elegir lo más fácil por pereza, sino reconocer que ahí está el mayor desajuste entre lo que se midió y lo que se concluyó de esa medición.

Referencias

- Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K., & Thorne, S. (Eds.). (2018). The site reliability workbook: Practical ways to implement SRE. O'Reilly Media.
- Majors, C., Fong-Jones, L., & Miranda, G. (2022). Observability engineering: Achieving production excellence. O'Reilly Media.
- Parker, A., Spoonhower, D., Mace, J., & Isaacs, R. (2020). Distributed tracing in practice: Instrumenting, analyzing, and debugging microservices. O'Reilly Media.
- Universidad de La Salle. (2026). Recurso educativo digital 2: Pruebas y rendimiento de carga [Material del curso]. Calidad en Software II.